

Лабораторная работа № 4. Генерация случайных чисел

Теоретические основы

Современная информатика широко использует псевдослучайные числа в самых разных приложениях — от метода Монте-Карло и имитационного моделирования до криптографии. При этом от качества используемых ГПСЧ напрямую зависит качество получаемых результатов.

В теории вероятностей случайная величина имеет **дискретное равномерное распределение**, если она принимает конечное число значений с равными вероятностями.

Никакой детерминированный алгоритм не может генерировать полностью случайные числа, а только лишь аппроксимировать некоторые свойства случайных чисел.

Любой ГПСЧ с ограниченными ресурсами рано или поздно закикливается. Длина циклов ГПСЧ зависит от самого генератора и в среднем составляет около $2^{n/2}$, где n — это размер внутреннего состояния в битах. Если ГПСЧ может сходиться к слишком коротким циклам, такой ГПСЧ становится предсказуемым и является непригодным.

Большинство простых арифметических генераторов хотя и обладают большой скоростью, но страдают от многих серьёзных недостатков:

Слишком короткий период/периоды

Последовательные значения не являются независимыми

Некоторые биты «менее случайны», чем другие

Неравномерное одномерное распределение

Обратимость.

В качестве примеров эффективно реализуемых на компьютерах алгоритмов генерации случайных последовательностей можно привести четыре генератора с периодом $T_{\max} \approx 2^{96}$:

Таблица 1.

$x_{t+1} = (1176x_t + 1476x_{t-1} + 1776x_{t-2}) \bmod (2^{32} - 5);$	(1)
$x_{t+1} = 2^{13}(x_t + x_{t-1} + x_{t-2}) \bmod (2^{32} - 5);$	(2)
$x_{t+1} = (1995x_t + 1998x_{t-1} + 2001x_{t-2}) \bmod (2^{35} - 849);$	(3)
$x_{t+1} = 2^{19}(x_t + x_{t-1} + x_{t-2}) \bmod (2^{35} - 1629).$	(4)

Инструментальные средства

Батарея тестов Диехарда (DIEHARD.EXE)

Эти тесты требуют, чтобы на вход был подан большой файл (более 11 МБ), который содержит случайные целые двоичные числа.

Большинство тестов возвращает p -значение, которое должно быть однородно на $[0,1)$, если входной файл содержит действительно независимые случайные биты. Эти p -значения получены из выражений $p=F(X)$, где F — принятое распределение случайной переменной X . При этом предполагается, что F задает только асимптотическое приближение, для которого пригодными значениями будут и худшие на концах интервала. Таким образом, возможны случайные p -значения около 0 или 1, типа 0.0012 или 0.9983.

Когда часть входного потока действительно не случайна, Вы получите значение p близкое к 0 или 1 в шести или более местах. Поэтому, если $p < 0.025$ или $p > 0.975$ не

следует делать вывод, что датчик "провалил испытание на доверительном уровне вероятности 0.05".

Использование:

1. запустите программу DIEHARD.EXE;
2. введите имя файла, содержащего двоичные данные (размер файла должен быть не менее 11 МБ);
3. введите имя текстового файла, в который программа должна поместить результат;
4. введите подряд 15 символов «1», означающих, что вы хотите выполнить соответствующие 15 тестов;
5. После окончания работы программы просмотрите полученный текстовый файл и оцените результаты.

Пример:

Первый тест выдал:

BIRTHDAY SPACINGS TEST

```
...
The 9 p-values were
    .242155    .215332    .409381    .354095    .872947
    .233062    .311486    .154540    .862131
A KSTEST for the 9 p-values yields .732411
```

Все результаты находятся в пределах $0.025 < p < 0.975$, следовательно, последовательность прошла испытания на случайность для этого теста.

```
This is the MINIMUM DISTANCE test
for random integers in the file out.bin
Sample no.    d^2      avg      equiv uni
    5      1.5952    1.5472    .798746
   10      1.2017    1.2026    .701124
   15      3.9543    1.4901    .981206
   20      .1644    1.4644    .152315
   25      .1076    1.4183    .102472
   30      1.3152    1.2843    .733342
   35      1.5188    1.2278    .782682
   40      1.8259    1.3241    .840396
   45      1.3176    1.3282    .733998
   50      .0866    1.2443    .083322
   55      1.4631    1.2583    .770168
   60      .7246    1.2362    .517238
   65      2.9113    1.2086    .946385
   70      .7005    1.2381    .505427
   75      .1027    1.2186    .098026
   80      .5791    1.1725    .441213
   85      .6604    1.1690    .485088
   90      .6227    1.1295    .465185
   95      1.7329    1.1027    .824757
  100      .8229    1.1220    .562677
MINIMUM DISTANCE TEST for out.bin
Result of KS test on 20 transformed mindist^2's:
p-value= .639196
```

Хотя в одном случае $p = 0.981206$ не попадает в интервал $0.025 < p < 0.97$, тем не менее, в остальных случаях результаты находятся в нужных пределах, следовательно, последовательность прошла испытания на случайность для этого теста.

и. т. д.

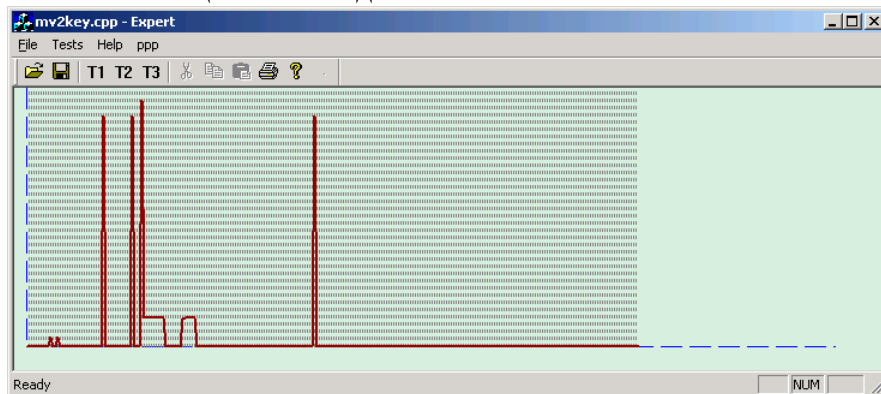
Если последовательность прошла испытания для всех тестов батареи, то можно сделать вывод, что датчик генерирует качественную псевдослучайную последовательность.

Если последовательность не прошла хотя бы один из тестов, то можно сделать вывод, что последовательность не является случайной.

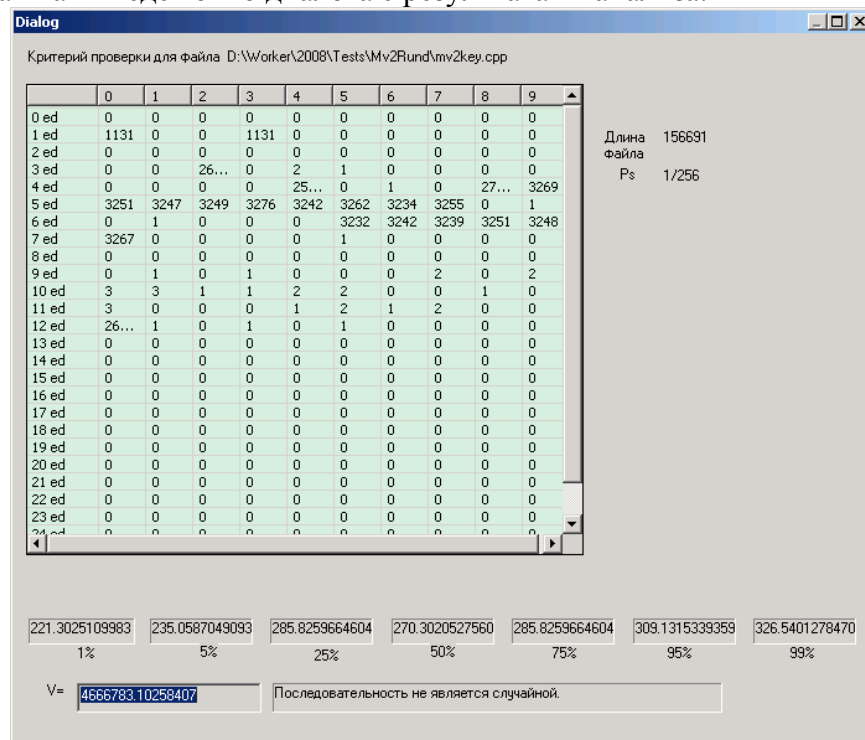
Программа Expert.exe

Эта программа реализует проверку заданного файла на случайность в соответствии с простейшим критерием «Хи-квадрат» для 256 степеней свободы.

Программа может проанализировать любой файл и предназначена для быстрой оценки случайности имеющейся последовательности.



Для использования программы откройте файл и нажмите на кнопку T1. Программа выведет окно диалога с результатами анализа.



Программа выдает три значения:

Последовательность не является случайной

Последовательность подозрительна

Последовательность почти подозрительна

Результаты «Последовательность подозрительна» или «Последовательность почти подозрительна» говорят о том, что выбранный файл содержит похожую на случайную последовательность символов. В этом случае необходимо использовать более мощные тесты для оценки того является ли сохраненная в выбранном файле последовательность достаточно случайной.

Задание для самостоятельной работы

1. Пользуясь программой «Expert» проверить случайность файлов содержащих программный код (.exe), архивы (.rar, .zip, и т.д.).

Сформулировать выводы.

2. Используя программные средства (MS Visual Studio, Borland, и т. д.) написать программу, использующую для генерации датчик псевдослучайных чисел, встроенный в эти средства.

Пример для C++:

```
#define BUFOUTSIZE 20000000

int APIENTRY WinMain(HINSTANCE hInstance,
                    HINSTANCE hPrevInstance,
                    LPSTR lpCmdLine,
                    int nCmdShow)
{
    FILE * out;
    DWORD i, dwRand;
    BYTE * hMem = 0;
    try{
        out = fopen("rand_out.bin","wb");
        if (out == NULL) return -1;
        hMem = (BYTE *) ::VirtualAlloc(0, BUFOUTSIZE, MEM_COMMIT,PAGE_READWRITE);
        if (hMem){
            srand(0);
            for (i= 0; i< BUFOUTSIZE; i++) {
                dwRand = rand();
                memcpy( hMem + i, (BYTE *) &dwRand, 1);
            }
            fwrite(hMem,1, BUFOUTSIZE,out);
            ::VirtualFree(hMem,0,MEM_RELEASE);
        }
    }
    catch(...){
        printf("\r\n\error create file");
    }
    fclose(out);
    return 0;
}
```

Выполнить анализ полученной последовательности.

Сделать вывод о качестве встроенного генератора.

3. Используя функцию CryptoAPI (<wincrypt.h>)¹ CryptGenRandom сгенерировать файл содержащий псевдослучайные последовательности:

Пример для C++:

```
DWORD cbGoop = 1000;
BYTE lpGoop[1000];

HCRYPTPROV m_hProv;
m_hProv = NULL;
::CryptAcquireContext(&m_hProv, NULL,NULL, PROV_RSA_FULL,CRYPT_VERIFYCONTEXT);
if (m_hProv == NULL)
    throw GetLastError();
if (!CryptGenRandom(m_hProv, cbGoop, lpGoop))
    printf("\r\n\ошибка генерации данных");
if (m_hProv) ::CryptReleaseContext(m_hProv, 0);
```

Выполнить анализ полученной последовательности.

Сделать вывод о качестве криптографического генератора.

4. Самостоятельно реализовать один из генераторов приведенных в таблице 1. Используя реализованный датчик сгенерировать файл содержащий псевдослучайные последовательности.

Выполнить анализ полученной последовательности.

Сделать вывод о качестве генератора.

Оформить отчет.

¹ для MS VS 6.0 возможно потребуется определить макрос #define _WIN32_WINNT 0x0500